

핸즈온 랩: Microsoft Fabric에서 Data Wrangler로 데이터 전처리하기

이 랩에서는 Microsoft Fabric에서 **Data Wrangler**를 사용하여 데이터를 전처리하고, 일반적인 데이터 과학 작업 라이브러리를 사용하여 코드를 생성하는 방법을 배웁니다.

개념 설명: Data Wrangler

Data Wrangler는 Microsoft Fabric에 내장된 데이터 준비 및 정제 도구입니다. 코드를 한 줄도 작성하지 않고도 시각적인 인터페이스를 통해 데이터를 탐색하고, 정리하며, 변환하는 복잡한 작업을 수행할 수 있습니다. 사용자가 UI에서 수행한 각 변환 단계는 자동으로 재사용 가능한 Python 코드로 생성되어 Notebook에 추가됩니다. 이는 데이터 과학자들이 모델링에 앞서 가장 많은 시간을 소요하는 데이터 전처리(preprocessing) 작업을 획기적으로 가속화하고, 작업의 재현성을 보장하는 강력한 도구입니다.

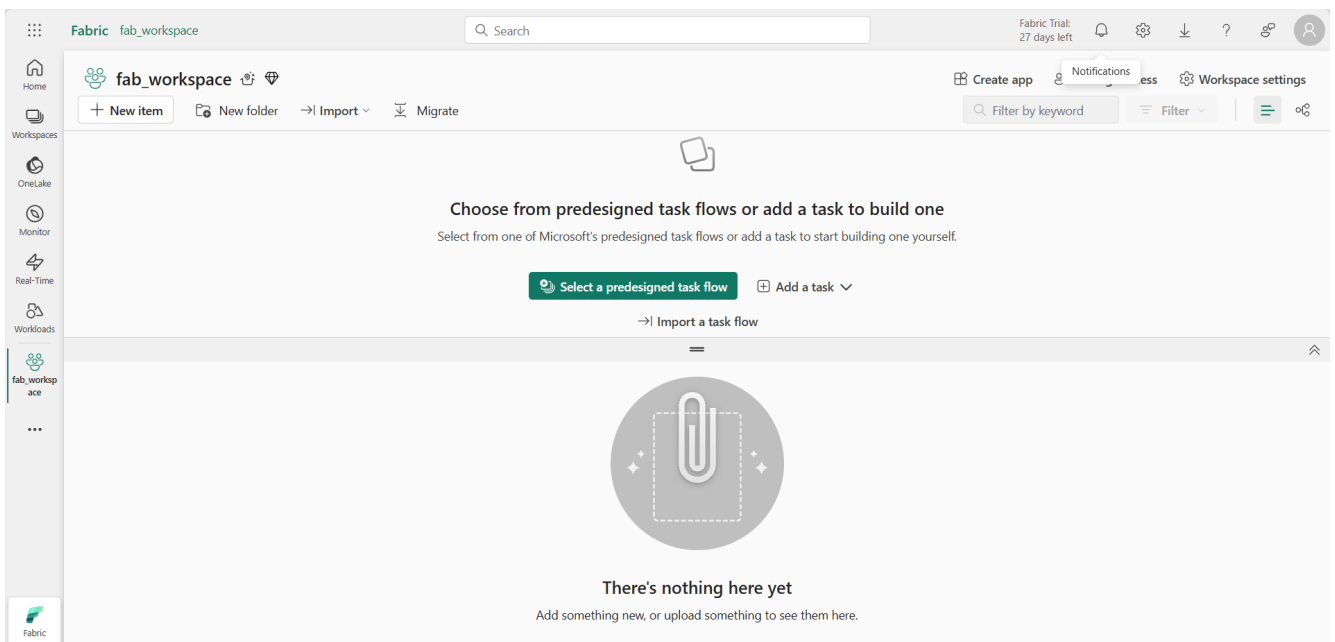
이 실습을 완료하는 데 약 **30분**이 소요됩니다.

참고: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 평가판이 활성화된 Workspace를 만들어야 합니다.

- 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) `https://app.fabric.microsoft.com/home?experience=fabric` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
- 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
- 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial, Premium, 또는 Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
- 새 Workspace가 열리면 처음에는 비어 있어야 합니다.



Notebook 만들기

1. 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Science* 섹션에서 **Notebook**을 선택하고, 원하는 고유한 이름을 지정합니다.
2. 첫 번째 셀을 Markdown 셀로 변환한 다음, 내용을 삭제하고 다음 텍스트를 입력합니다.

```
# Perform data exploration for data science

Use the code in this notebook to perform data exploration for data science.
```

데이터프레임으로 데이터 로드하기

이제 코드를 실행하여 데이터를 가져올 준비가 되었습니다. Azure Open Datasets의 [OJ Sales 데이터 세트](#)를 사용합니다. 데이터를 로드한 후에는 Data Wrangler에서 지원하는 구조인 Pandas 데이터프레임으로 변환합니다.

1. 새 코드 셀을 추가하고 다음 코드를 입력하여 데이터 세트를 로드합니다.

```
# Azure storage access info for open dataset diabetes
blob_account_name = "azureopendatastorage"
blob_container_name = "ojsales-simulatedcontainer"
blob_relative_path = "oj_sales_data"
blob_sas_token = r"" # Blank since container is Anonymous access

# Set Spark config to access blob storage
wasbs_path = f"wasbs://{s}@{s}.blob.core.windows.net/{s}" % (blob_container_name,
blob_account_name, blob_relative_path)
spark.conf.set("fs.azure.sas.{s}.{s}.blob.core.windows.net" % (blob_container_name,
blob_account_name), blob_sas_token)
print("Remote blob path: " + wasbs_path)

# Spark reads csv
df = spark.read.csv(wasbs_path, header=True)
```

2. 셀을 실행합니다.
3. 새 코드 셀을 추가하고 다음 코드를 실행하여 Spark 데이터프레임을 Pandas 데이터프레임으로 변환하고, 데이터 타입을 정리합니다.

```
import pandas as pd

df = df.toPandas()
df = df.sample(n=500, random_state=1) # 데이터가 크므로 500개의 샘플만 사용

# 각 열의 데이터 타입을 올바르게 설정
df['WeekStarting'] = pd.to_datetime(df['WeekStarting'])
df['Quantity'] = df['Quantity'].astype('int')
df['Advert'] = df['Advert'].astype('int')
df['Price'] = df['Price'].astype('float')
df['Revenue'] = df['Revenue'].astype('float')

df = df.reset_index(drop=True)
df.head(4)
```

요약 통계 보기

이제 데이터를 로드했으므로, 다음 단계는 Data Wrangler를 사용하여 전처리하는 것입니다.

1. Notebook 리본에서 **Data**를 선택한 다음, **Launch Data Wrangler** 드롭다운을 선택합니다.
2. **df** 데이터 세트를 선택합니다. Data Wrangler가 실행되면 **Summary** 패널에 데이터프레임의 기술적 개요가 생성됩니다.
3. **Revenue** 특징(feature)을 선택하고, 이 특징의 데이터 분포를 관찰합니다.
4. **Summary** 사이드 패널의 세부 정보를 검토하고 통계 값을 관찰합니다.

Summary	Revenue	▼
Data type	float64	
Rows	500	
Unique values	500	
Missing values	0	
▼ Statistics		
Mean	33,459.546	
Standard deviation	8,032.233	
Minimum	18,384.04	
25th percentile	27,234.435	
Median	32,892.74	
75th percentile	39,553.87	
Maximum	52,097.67	
▼ Advanced Statistics		
Kurtosis	-0.784	
Skew	0.218	

평균 수익은 약 **\$33,459.54**이며, 표준 편차는 **\$8,032.23**입니다. 이는 수익 값이 평균 주위로 약 **\$8,032.23** 범위에 걸쳐 퍼져 있음을 시사합니다.

텍스트 데이터 서식 지정하기

핸즈온의 의미: 이 단계에서는 코딩 없이 Data Wrangler의 UI를 사용하여 텍스트 데이터를 정제하는 과정을 경험합니다. 'Find and replace'와 'Capitalize' 같은 일반적인 텍스트 변환 작업을 마우스 클릭만으로 수행하고, 그 결과가 실시간으로 미리보기에 반영되는 것을 확인합니다. 마지막으로, 이 모든 시각적 작업이 재사용 가능한 Python 코드로 자동 생성되는 과정을 통해 Data Wrangler의 생산성을 체감하게 됩니다.

1. **Data Wrangler** 대시보드에서 그리드의 **Brand** 특징을 선택합니다.

2. **Operations** 패널에서 **Find and replace**를 확장하고, **Find and replace**를 선택합니다.
3. **Find and replace** 패널에서 다음 속성을 변경합니다.
 - **Old value:** "."
 - **New value:** " " (공백 문자)
4. **Apply**를 선택합니다.
5. 다시 **Operations** 패널로 돌아가 **Format**을 확장합니다.
6. **Capitalize first character**를 선택하고, **Capitalize all words** 토글을 켜 다음, **Apply**를 선택합니다.
7. **Add code to notebook**을 선택합니다. 생성된 코드가 자동으로 노트북 셀에 복사됩니다.
8. Data Wrangler에서 생성된 코드는 원본 데이터프레임을 덮어쓰지 않으므로, 10행과 11행을 `df = clean_data(df)` 코드로 바꿉니다. 최종 코드 블록은 다음과 같아야 합니다.

```
def clean_data(df):  
    # Replace all instances of "." with " " in column: 'Brand'  
    df['Brand'] = df['Brand'].str.replace(".", " ", case=False, regex=False)  
    # Capitalize the first character in column: 'Brand'  
    df['Brand'] = df['Brand'].str.title()  
    return df  
  
df = clean_data(df)
```

9. 코드 셀을 실행하고, `Brand` 변수를 확인합니다.

```
df['Brand'].unique()
```

결과를 *Minute Maid*, *Dominicks*, *Tropicana* 값을 보여야 합니다.

원-핫 인코딩(One-Hot Encoding) 변환 적용하기

개념 설명: One-Hot Encoding

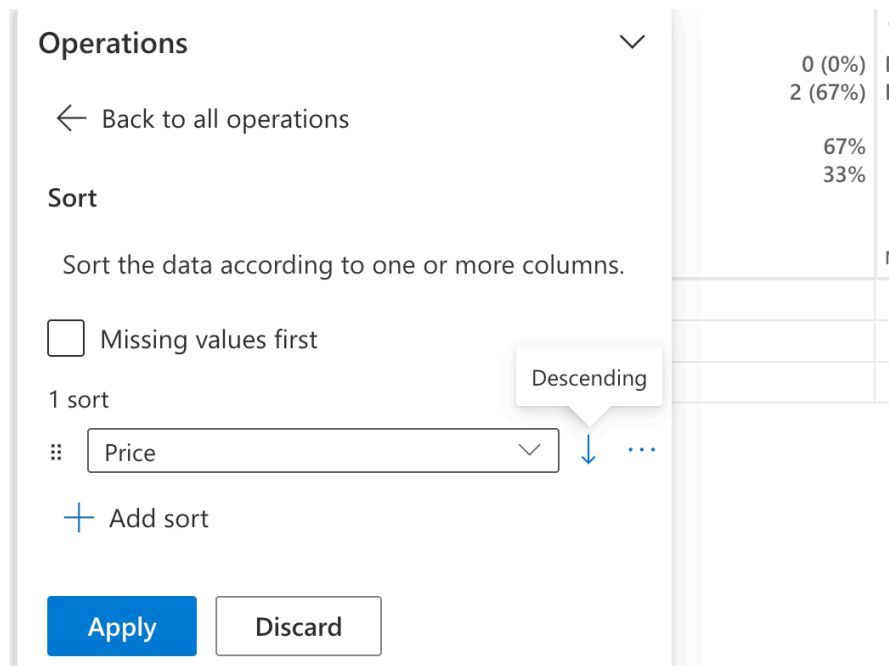
원-핫 인코딩은 범주형 변수(예: 'Brand' 열의 'Dominicks', 'Tropicana')를 머신러닝 모델이 이해할 수 있는 숫자 형식으로 변환하는 일반적인 기법입니다. 각 범주 값을 고유한 이진(0 또는 1) 열로 변환합니다. 예를 들어, 'Brand' 열은 `Brand_Dominicks`, `Brand_Tropicana`, `Brand_Minute Maid` 라는 세 개의 새 열로 변환됩니다. 특정 행의 브랜드가 'Dominicks'이면 `Brand_Dominicks` 열은 1이 되고 나머지 두 열은 0이 됩니다.

1. `df` 데이터프레임에 대해 Data Wrangler를 실행합니다.
2. 그리드에서 `Brand` 특징을 선택합니다.
3. **Operations** 패널에서 **Formulas**를 확장하고, **One-hot encode**를 선택합니다.
4. **One-hot encode** 패널에서 **Apply**를 선택합니다.
5. Data Wrangler 표시 그리드의 끝으로 이동합니다. `Brand_Dominicks`, `Brand_Minute Maid`, `Brand_Tropicana` 라는 세 개의 새 특징이 추가되고, 원래 `Brand` 특징은 제거된 것을 확인하세요.
6. 코드를 생성하지 않고 Data Wrangler를 종료합니다.

정렬 및 필터링 작업

특정 매장의 수익 데이터를 검토한 다음 제품 가격을 정렬해야 한다고 가정해 보겠습니다.

1. **df** 데이터프레임에 대해 Data Wrangler를 실행합니다.
2. **Operations** 패널에서 **Sort and filter**를 확장하고 **Filter**를 선택합니다.
3. **Filter** 패널에서 다음 조건을 추가합니다.
 - **Target column:** Store
 - **Operation:** Equal to
 - **Value:** 1227
 - **Action:** Keep matching rows
4. **Apply**를 선택합니다.
5. **Revenue** 특징을 선택하고 **Summary** 사이드 패널의 세부 정보를 검토합니다. 왜도(skewness)가 **-0.751**로, 약간의 왼쪽 치우침(음의 왜도)을 나타냅니다.
6. 다시 **Operations** 패널로 돌아가 **Sort and filter**를 확장하고, **Sort values**를 선택합니다.
7. **Sort values** 패널에서 다음 속성을 선택합니다.
 - **Column name:** Price
 - **Sort order:** Descending



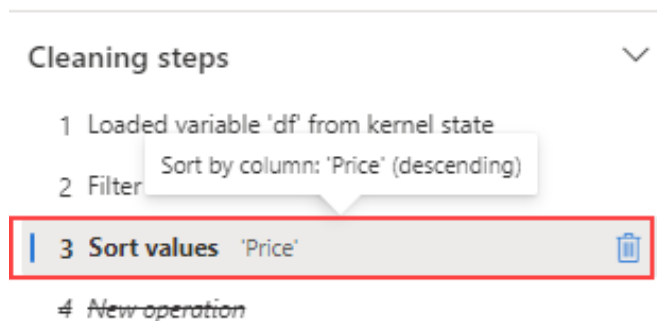
8. **Apply**를 선택합니다. 매장 **1227**의 가장 높은 제품 가격은 **\$2.68**임을 확인할 수 있습니다.

#	Price	#
6)	Missing:	0 (0%)
6)	Distinct:	3 (100%)
1	Min 2.0	Max 2.68
1		2.68
1		2.08
1		2.0

단계 찾아보기 및 제거하기

실수로 이전 단계에서 만든 정렬을 제거해야 한다고 가정해 봅시다.

1. **Cleaning steps** 패널로 이동합니다.
2. **Sort values** 단계를 선택합니다.
3. 삭제 아이콘을 선택하여 제거합니다.



중요: 그리드 뷰와 요약은 현재 단계에 제한됩니다. 변경 사항이 이전 단계인 **Filter** 단계로 되돌아가는 것을 확인하세요.

4. 코드를 생성하지 않고 Data Wrangler를 종료합니다.

데이터 집계하기

각 브랜드가 생성하는 평균 수익을 이해해야 한다고 가정해 보겠습니다.

1. `df` 데이터프레임에 대해 Data Wrangler를 실행합니다.
2. **Operations** 패널에서 **Group by and aggregate**를 선택합니다.
3. **Columns to group by** 패널에서 `Brand` 특징을 선택합니다.
4. **Add aggregation**을 선택하고, 집계할 열로 `Revenue` 를, 집계 유형으로 `Mean` 을 선택합니다.
5. **Apply**를 선택합니다.
6. **Copy code to clipboard**를 선택하고, 코드를 생성하지 않고 Data Wrangler를 종료합니다.

7. `Brand` 변환 코드와 집계 단계에서 생성된 코드를 `clean_data(df)` 함수에 결합합니다. 최종 코드 블록은 다음과 같아야 합니다.

```
def clean_data(df):  
    # Replace all instances of "." with " " in column: 'Brand'  
    df['Brand'] = df['Brand'].str.replace(".", " ", case=False, regex=False)  
    # Capitalize the first character in column: 'Brand'  
    df['Brand'] = df['Brand'].str.title()  
  
    # Performed 1 aggregation grouped on column: 'Brand'  
    df = df.groupby(['Brand']).agg(Revenue_mean=('Revenue', 'mean')).reset_index()  
  
    return df  
  
df = clean_data(df)
```

8. 셀 코드를 실행하고 데이터프레임의 데이터를 확인합니다.

```
print(df)  
#또는 display(df)
```

결과는 각 브랜드별 평균 수익을 보여줍니다.

	ABC Brand	1.2 Revenue_mean
1	Dominicks	33206.3309580838
2	Minute Maid	33532.9996319018
3	Tropicana	33637.8634117647

Notebook 저장 및 Spark 세션 종료하기

이제 모델링을 위한 데이터 전처리를 마쳤으므로, 의미 있는 이름으로 Notebook을 저장하고 Spark 세션을 종료할 수 있습니다.

1. Notebook 메뉴 바에서  **Settings** 아이콘을 사용하여 Notebook 설정을 봅니다.
2. Notebook의 **Name**을 **Preprocess data with Data Wrangler**로 설정한 다음 설정 창을 닫습니다.
3. Notebook 메뉴에서 **Stop session**을 선택하여 Spark 세션을 종료합니다.

리소스 정리

이 실습에서는 Notebook을 만들고 Data Wrangler를 사용하여 머신러닝 모델을 위한 데이터를 탐색하고 전처리했습니다.

전처리 단계 탐색을 마쳤다면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

1. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
2. 툴바의 ... 메뉴에서 **Workspace settings**를 선택합니다.
3. **General** 섹션에서 **Remove this workspace**를 선택합니다.